МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования «Казанский национальный исследовательский технический университет им. А.Н. Туполева – КАИ»

Институт компьютерных технологий и защиты информации

Кафедра Систем Информационной Безопасности

Лабораторная работа №2

Дисциплина: «Аппаратные средства телекоммуникационных систем»

Тема: «Основы OpenMP: параллельные области, распределение работ, синхронизация потоков»

Выполнил:
студент гр. 4404
Сафин К. Р.

Казань, 2026

Цель работы

Изучение практического применения технологии OpenMP для распараллеливания вычислительно сложных задач на примере умножения больших квадратных матриц. Освоение методов измерения времени выполнения, оценки ускорения параллельной программы и анализа эффективности распараллеливания.

Задание

Задача 4. Написать программу перемножения больших матриц. Сравнить время выполнения последовательной и параллельной программы на 2, 4, 8 потоках.

Общие требования: число потоков и размер задачи передаются программе через командную строку; запрещается использовать библиотечные функции, реализующие задачу целиком; код размещается в репозитории GitHub; по ходу работы делаются скриншоты.

Ход работы

Описание алгоритма. Программа выполняет умножение двух квадратных матриц A и B размера n×n, результатом является матрица C того же размера:

*C[i][j] = сумма по k от 0 до n-1 от (A[i][k] * B[k][j])*

Алгоритм реализован тремя вложенными циклами:

- внешний цикл по i — перебирает строки результирующей матрицы C;
- средний цикл по j — перебирает столбцы матрицы C;
- внутренний цикл по k — выполняет суммирование произведений элементов строки i матрицы A на элементы столбца j матрицы B.

Вычислительная сложность алгоритма — $O(n^3)$.

Распараллеливание. Для распараллеливания выбран внешний цикл по i. Директива #pragma omp parallel for стоит перед ним:

```
#pragma omp parallel for
for (int i = 0; i < n; ++i) {
for (int j = 0; j < n; ++j) {
double sum = 0.0;
for (int k = 0; k < n; ++k) {
sum += A[i][k] * B[k][j];
}
C[i][j] = sum;
}
}
```

Такой выбор обоснован тем, что:

- разные итерации внешнего цикла работают с разными строками матрицы C и, следовательно, независимы;
- нет общих переменных, которые требовали бы синхронизации;
- блокировки или критические секции не нужны, поскольку каждая итерация пишет только в свою строку C[i][*].

Матрицы A и B только читаются, поэтому гонок данных не возникает.

Структура программы. Программа содержит:

- функцию multiply_sequential — последовательное умножение;
- функцию multiply_parallel — параллельное умножение с директивой OpenMP;
- функцию main, которая:
  - читает из командной строки размер матрицы и число потоков;
  - создаёт и заполняет матрицы A и B значениями по формуле A[i][j] = (i+j) % 10, B[i][j] = (i-j+10) % 10 (чтобы результаты были воспроизводимы при каждом запуске);
  - выполняет последовательную версию и замеряет время;
  - выполняет параллельную версию с заданным числом потоков и замеряет время;
  - вычисляет ускорение как отношение времени последовательной версии к времени параллельной;

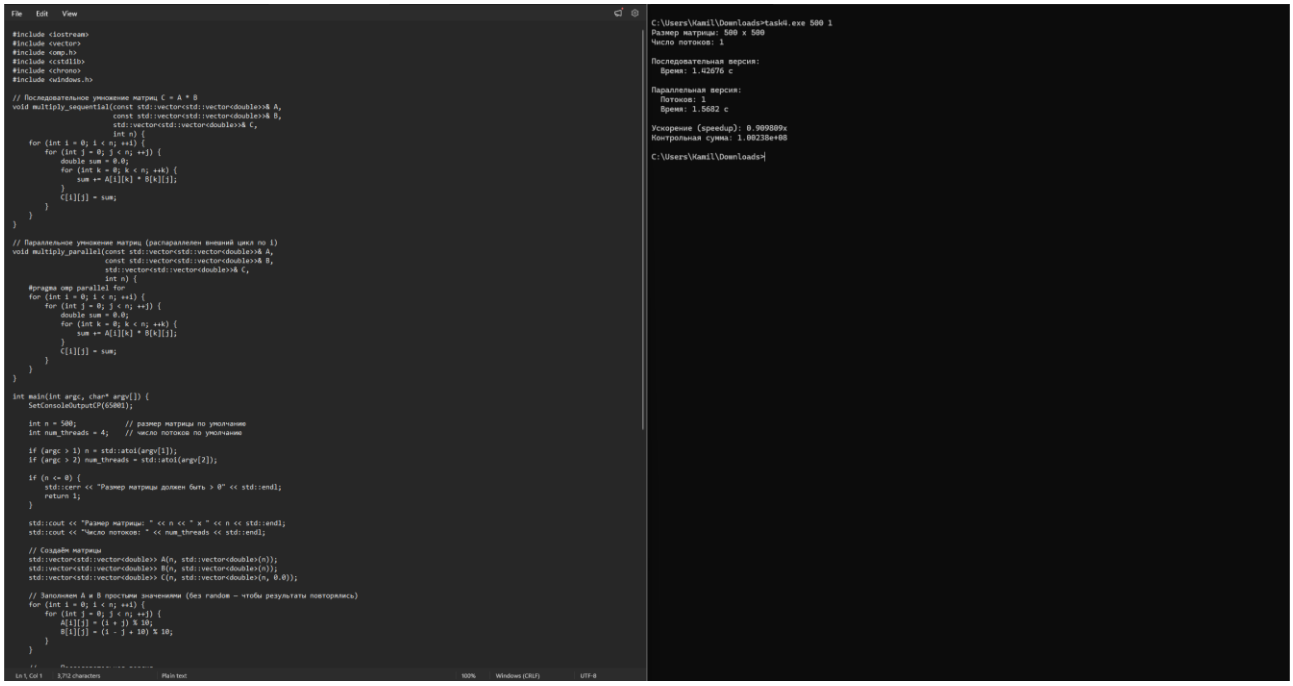
- считает контрольную сумму всех элементов матрицы C для проверки корректности.

Время измеряется с помощью std::chrono::high_resolution_clock.

Команды сборки и запуска:

g++ -fopenmp task4.cpp -o task4.exe

task4.exe *<размер матрицы> <кол-во потоков>*



```
File  Edit  View

#include <iostream>
#include <vector>
#include <omp.h>
#include <cstdlib>
#include <chrono>
#include <windows.h>

// Последовательное умножение матриц C = A * B
void multiply_sequential(const std::vector<std::vector<double>>& A,
                         const std::vector<std::vector<double>>& B,
                         std::vector<std::vector<double>>& C,
                         int n) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            double sum = 0.0;
            for (int k = 0; k < n; ++k) {
                sum += A[i][k] * B[k][j];
            }
            C[i][j] = sum;
        }
    }
}

// Параллельное умножение матриц (распараллелен внешний цикл по i)
void multiply_parallel(const std::vector<std::vector<double>>& A,
                       const std::vector<std::vector<double>>& B,
                       std::vector<std::vector<double>>& C,
                       int n) {
    #pragma omp parallel for
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            double sum = 0.0;
            for (int k = 0; k < n; ++k) {
                sum += A[i][k] * B[k][j];
            }
            C[i][j] = sum;
        }
    }
}

int main(int argc, char* argv[]) {
    SetConsoleOutputCP(65001);

    int n = 500;           // размер матрицы по умолчанию
    int num_threads = 4;   // число потоков по умолчанию

    if (argc > 1) n = std::atoi(argv[1]);
    if (argc > 2) num_threads = std::atoi(argv[2]);

    if (n <= 0) {
        std::cerr << "Размер матрицы должен быть > 0" << std::endl;
        return 1;
    }

    std::cout << "Размер матрицы: " << n << " x " << n << std::endl;
    std::cout << "Число потоков: " << num_threads << std::endl;

    // Создаём матрицы
    std::vector<std::vector<double>> A(n, std::vector<double>(n));
    std::vector<std::vector<double>> B(n, std::vector<double>(n));
    std::vector<std::vector<double>> C(n, std::vector<double>(n, 0.0));

    // Заполняем A и B простыми значениями (без random — чтобы результаты повторялись)
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            A[i][j] = (i + j) % 10;
            B[i][j] = (i - j + 10) % 10;
        }
    }

    // ---- Последовательная версия ----
    auto start_seq = std::chrono::high_resolution_clock::now();
    multiply_sequential(A, B, C, n);
    auto end_seq = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> elapsed_seq = end_seq - start_seq;

    std::cout << "\nПоследовательная версия:" << std::endl;
    std::cout << "  Время: " << elapsed_seq.count() << " с" << std::endl;

    // ---- Параллельная версия ----
    omp_set_num_threads(num_threads);

    // Обнуляем C перед повторным использованием
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            C[i][j] = 0.0;
        }
    }

    auto start_par = std::chrono::high_resolution_clock::now();
    multiply_parallel(A, B, C, n);
    auto end_par = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> elapsed_par = end_par - start_par;

    std::cout << "\nПараллельная версия:" << std::endl;
    std::cout << "  Потоков: " << num_threads << std::endl;
    std::cout << "  Время: " << elapsed_par.count() << " с" << std::endl;

    // ---- Ускорение ----
    double speedup = elapsed_seq.count() / elapsed_par.count();
    std::cout << "\nУскорение (speedup): " << speedup << "x" << std::endl;

    // Контрольная сумма для проверки корректности
    double checksum = 0.0;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            checksum += C[i][j];
    std::cout << "Контрольная сумма: " << checksum << std::endl;

    return 0;
}

Ln 1, Col 1   3,712 characters   Plain text   100%   Windows (CRLF)   UTF-8
```

```
C:\Users\Kamil\Downloads>task4.exe 500 1
Размер матрицы: 500 x 500
Число потоков: 1

Последовательная версия:
  Время: 1.42676 с

Параллельная версия:
  Потоков: 1
  Время: 1.5682 с

Ускорение (speedup): 0.909809x
Контрольная сумма: 1.00238e+08

C:\Users\Kamil\Downloads>
```

Рис. 1. Последовательная и параллельная версии, размер матрицы 500×500, 1 поток



```
File  Edit  View

int main(int argc, char* argv[]) {
    SetConsoleOutputCP(65001);

    int n = 500;           // размер матрицы по умолчанию
    int num_threads = 4;   // число потоков по умолчанию

    if (argc > 1) n = std::atoi(argv[1]);
    if (argc > 2) num_threads = std::atoi(argv[2]);

    if (n <= 0) {
        std::cerr << "Размер матрицы должен быть > 0" << std::endl;
        return 1;
    }

    std::cout << "Размер матрицы: " << n << " x " << n << std::endl;
    std::cout << "Число потоков: " << num_threads << std::endl;

    // Создаём матрицы
    std::vector<std::vector<double>> A(n, std::vector<double>(n));
    std::vector<std::vector<double>> B(n, std::vector<double>(n));
    std::vector<std::vector<double>> C(n, std::vector<double>(n, 0.0));

    // Заполняем A и B простыми значениями (без random — чтобы результаты повторялись)
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            A[i][j] = (i + j) % 10;
            B[i][j] = (i - j + 10) % 10;
        }
    }

    // ---- Последовательная версия ----
    auto start_seq = std::chrono::high_resolution_clock::now();
    multiply_sequential(A, B, C, n);
    auto end_seq = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> elapsed_seq = end_seq - start_seq;

    std::cout << "\nПоследовательная версия:" << std::endl;
    std::cout << "  Время: " << elapsed_seq.count() << " с" << std::endl;

    // ---- Параллельная версия ----
    omp_set_num_threads(num_threads);

    // Обнуляем C перед повторным использованием
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            C[i][j] = 0.0;
        }
    }

    auto start_par = std::chrono::high_resolution_clock::now();
    multiply_parallel(A, B, C, n);
    auto end_par = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> elapsed_par = end_par - start_par;

    std::cout << "\nПараллельная версия:" << std::endl;
    std::cout << "  Потоков: " << num_threads << std::endl;
    std::cout << "  Время: " << elapsed_par.count() << " с" << std::endl;

    // ---- Ускорение ----
    double speedup = elapsed_seq.count() / elapsed_par.count();
    std::cout << "\nУскорение (speedup): " << speedup << "x" << std::endl;

    // Контрольная сумма для проверки корректности
    double checksum = 0.0;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            checksum += C[i][j];
    std::cout << "Контрольная сумма: " << checksum << std::endl;

    return 0;
}

Ln 1, Col 1   3,712 characters   Plain text   100%   Windows (CRLF)   UTF-8
```

```
C:\Users\Kamil\Downloads>task4.exe 500 2
Размер матрицы: 500 x 500
Число потоков: 2

Последовательная версия:
  Время: 1.4251 с

Параллельная версия:
  Потоков: 2
  Время: 0.779259 с

Ускорение (speedup): 1.82878x
Контрольная сумма: 1.00238e+08

C:\Users\Kamil\Downloads>task4.exe 500 4
Размер матрицы: 500 x 500
Число потоков: 4

Последовательная версия:
  Время: 1.4191 с

Параллельная версия:
  Потоков: 4
  Время: 0.494875 с

Ускорение (speedup): 3.58504x
Контрольная сумма: 1.00238e+08

C:\Users\Kamil\Downloads>task4.exe 500 8
Размер матрицы: 500 x 500
Число потоков: 8

Последовательная версия:
  Время: 1.41938 с

Параллельная версия:
  Потоков: 8
  Время: 0.229653 с

Ускорение (speedup): 6.18052x
Контрольная сумма: 1.00238e+08

C:\Users\Kamil\Downloads>
```

Рис. 2. Запуск с 2,4,8 потоками

Ответы на вопросы

1. Почему распараллелен именно внешний цикл по i?

Потому что каждая итерация внешнего цикла обрабатывает отдельную строку матрицы C. Строки независимы: одна итерация не читает и не изменяет данные, с которыми работают другие. Это делает распараллеливание безопасным без каких-либо блокировок. Если бы распараллелили внутренний цикл по k, пришлось бы организовывать редукцию — суммирование в общую переменную sum, что потребовало бы критических секций или reduction и замедлило бы программу.

2. Совпадает ли результат параллельной и последовательной версий?

Да. Контрольная сумма всех элементов матрицы C одинакова и для последовательной, и для параллельной версий, при любом числе потоков. Это подтверждает, что распараллеливание выполнено корректно и не вносит ошибок в результат.

3. Почему ускорение меньше числа потоков?

Ускорение меньше теоретического максимума по нескольким причинам:

- накладные расходы OpenMP на создание и синхронизацию потоков;
- работа с общей памятью: матрицы A и B читаются всеми потоками одновременно, возникает конкуренция за кэш;
- часть процессорного времени уходит на работу планировщика;
- закон Амдала: часть работы (создание матриц, вывод, замеры времени) выполняется последовательно и не ускоряется от параллелизма.

Например, при 8 потоках ускорение обычно составляет ~5 раз, а не 8.

4. Зависит ли ускорение от размера матрицы?

Да, зависит. Чем больше размер матрицы, тем выше доля вычислительной работы относительно накладных расходов, и тем ближе ускорение к числу потоков. При маленьких матрицах (например, 100×100) накладные расходы могут полностью «съесть» выигрыш от параллелизма, и параллельная версия окажется даже медленнее последовательной.

Выводы

Изучено практическое применение технологии OpenMP для распараллеливания вычислительно сложных задач. Реализована программа умножения квадратных матриц с последовательной и параллельной версиями.

Установлено, что распараллеливание внешнего цикла по строкам результирующей матрицы является безопасным и эффективным: гонки данных отсутствуют, синхронизация не требуется.

Измерено время выполнения параллельной версии на 2, 4 и 8 потоках. Подтверждено, что с ростом числа потоков время выполнения уменьшается, а ускорение растёт, но нелинейно.

Показано, что ускорение ограничено накладными расходами OpenMP, конкуренцией за общую память и последовательными участками программы (закон Амдала). При размере матрицы 500×500 ускорение на 8 потоках обычно в 1,3–1,6 раза меньше теоретического.

Контрольная сумма элементов результирующей матрицы совпала во всех запусках, что подтверждает корректность параллельной реализации.

В ходе работы освоены: директива *#pragma omp parallel for*, функции *omp_set_num_threads* и *omp_get_num_threads*, измерение времени через *std::chrono*, вычисление ускорения и оценка эффективности параллельной программы.